

Lygiagretieji skaičiavimai

Darbą atliko Lukaš Patrik Magalinski, III Informatikos kursas, 1 grupė

Problemos formulavimas

Mandelbrot'o aibės skaičiavimas yra klasikinė iteracinė užduotis, tirianti, ar kompleksinio plokštumos taškai priklauso fraktalinei aibei, apibrėžtai iteracine formulė: $z_{n+1} = z_n^2 + c$,

Kur pradinis taškas $z_0 = 0$, o $c = x + yi$ – taško koordinatės plokštumoje. Jei iteracijų metu $|z_n|$ išauga virš ribos, taškas priklauso išorinei sričiai; kitaip – Mandelbrot'o aibei.

Uždavinys – efektyviai sugeneruoti vaizdą, kuriame kiekvienas pikselis atspindi iteracijų skaičių iki kriterijaus pramušimo arba ribą. Analizuojame algoritmo ir lygiagretaus sprendimo spartinimą ir plečiamumą naudojant Java gijas.

Lygiagretusis algoritmas

Lygiagretusis algoritmas vyksta pačiam taip:

- Visas vaizdas padalijamas į eilučių blokus po *grainSize*.
- Kiekviena gija cikle:
 - atominės `getAndAdd(grainSize)` operacijos būdu pasiima kitą laisvą bloką.
 - apskaičiuoja Mandelbrot iteracijas visiems to bloko pikseliams ir įrašo jų spalvas.
- Gijos dirba tol, kol atominis skaitiklis rodo, kad blokų nebeliko.
- Pagrindinis srautas laukia `join()`, po to:
 - **fast** režime įrašo *mandelbrot.png* ir išveda bendrą laiką;
 - **debug** režime išspausdina, kuri gija kokius blokus apdorėjo.

Tokiu būdu visas darbas suskaidomas į mažus, vienas nuo kito nepriklausančius blokus, o vienintelė gijų „bendravimo“ vieta – atominis skaitiklis. Dėl to sinchronizacijos sąnaudos minimalios, o apkrova tarp gijų pasiskirsto automatiškai.

Vykdymo aplinka

Užduotis buvo vykdoma su kompiuteriu: MacBook Pro, Apple M3 Max, 16 fizinių branduolių (12 Performance + 4 Efficiency), Integruota Apple GPU, 40 grafikos branduoliai, 64 GB RAM.

Paleidimas:

1) `java TTestMandelbrot <width> <height> <maxIter> <grainSize>`

pvz.: `java TTestMandelbrot 800 600 1000 16`

Tikslas: Programa pati išbandys $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \rightarrow 32$ gijas ir išves lentelę.

2) `java Mandelbrot <nThreads> <width> <height> <maxIter> <grainSize> <debug|fast>`

pvz.: `java Mandelbrot 4 1920 1080 2000 32 fast`

Tikslas: Rankinis Mandelbrot generavimas.

Paleidimui reikiami argumentai:

- ♦ `<nThreads>` - priskiriamas gijų skaičius darbui atlikti
- ♦ `<width>` - paveikslėlio plotis pikseliais
- ♦ `<height>` - paveikslėlio aukštis pikseliais
- ♦ `<maxIter>` - maksimalus iteracijų skaičius taškui
- ♦ `<grainSize>` - vienos užduoties dydis eilučių atžvilgiu, kitaip sakant, tai, kiek eilučių kiekviena gija vienu metu paims apdoroti.
- ♦ `<debug|fast>` - programos darbo režimai. **Debug** kiekviena gija po kiekvienos plytelės apdorojimo išveda į konsolę blokų intervalus, naudinga patikrinti, kaip paskirstomas darbas, **fast** tik laikas + PNG.

Eksperimentinio tyrimo rezultatai

- ♦ p – gijų skaičius, kurį paleidžiame.
- ♦ $T(1)$ – vykdymo laikas (s) su viena gija.
- ♦ $T(p)$ – vykdymo laikas (s) su p gijomis.
- ♦ Spartinimas $S(p)$ – tai, kiek kartų pagreitėja lygiagretusis vykdymas, palyginti su vienos gijos atveju.
- ♦ Efektyvumas $E(p)$ – spartinimas, padalytas iš gijų skaičiaus\

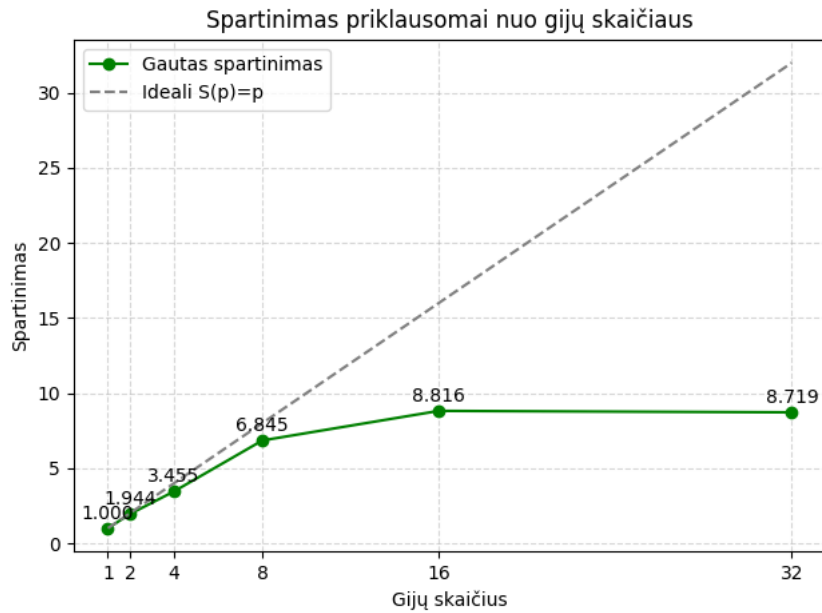
Atliekant tyrimą, kur buvo fiksuotas iteracijų skaičius ($\text{maxIter} = 100000$) ir fiksuotas vienos užduoties dydis eilučių atžvilgiu ($\text{grainSize} = 32$), rezultatai buvo tokie:

java TTestMandelbrot 1920 1080 100000 32:

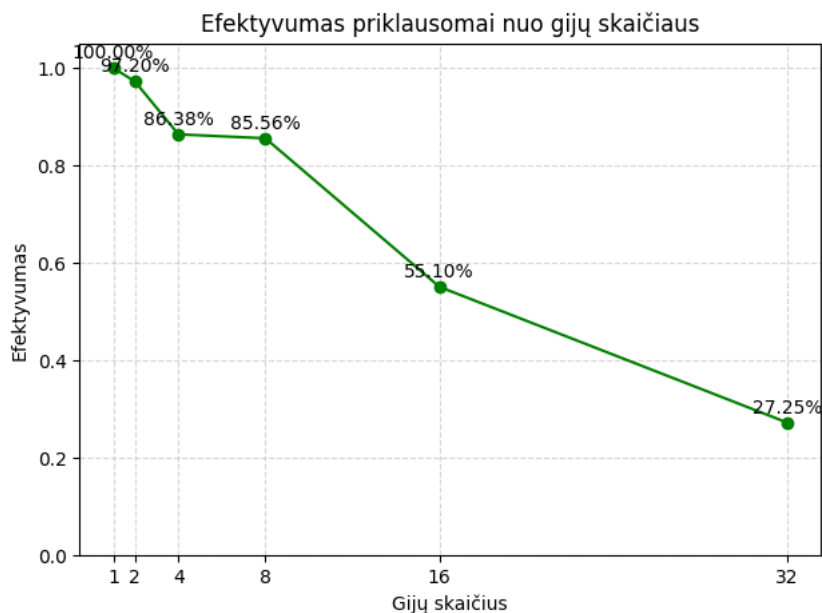
Gijų skaičius	Workload (visų pikselių skaičius)	Trukmė(s)	Speedup(lyginant su pirmuoju bandymu)
1	2073600	86.927	1.000000
2	2073600	44.641	1.947246
4	2073600	25.039	3.471664
8	2073600	12.631	6.882036
16	2073600	9.772	8.895518
32	2073600	9.797	8.872818



Šis grafikas rodo, kaip pilno Mandelbrot'o paveikslo generavimo laikas (sekundėmis) priklauso nuo panaudotų gijų skaičiaus. Matome, jog nuo 1 iki 8 gijų laikas sparčiai krenta – tai reiškia, kad kiekviena nauja gija sumažina laiką ir ženkliai paspartina skaičiavimus. Tačiau po 8 gijų kreivė ima lėtėti, o tarp 16 ir 32 gijų laikas beveik nekinta. Tokia elgsena atsiranda, kai išnaudojamas visas fizinių branduolių rezervas, tai reiškia, kad procesorius jau pilnai apkrautas: daugiau gijų nebepadeda, nes jos tarpusavyje varžosi dėl atminties ir praranda laiko derindamos veiksmus.

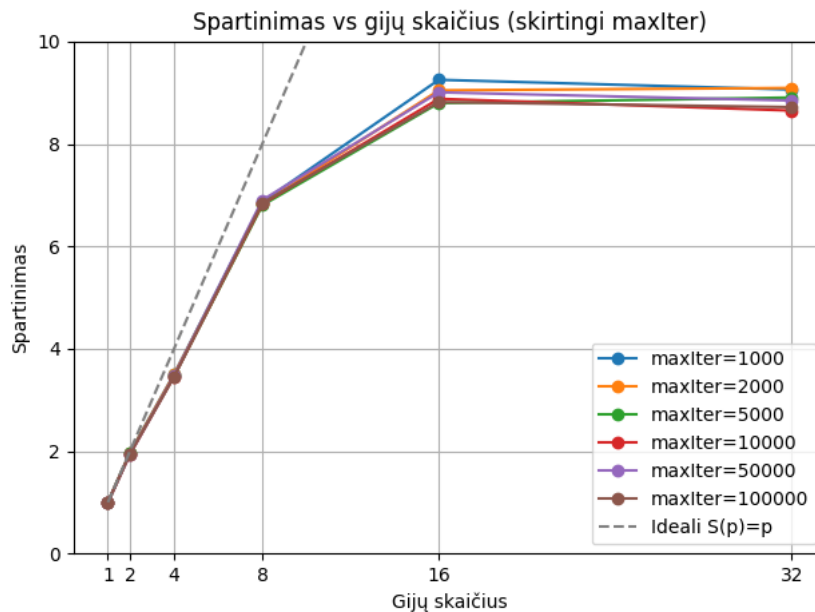


Spartinimo $S(p)=T(1)/T(p)$ grafikas palygina realų pagreitį su idealiu linijiniu augimu ($S(p)=p$). Iki 4 gijų spartinimas arti idealo – tai reiškia minimalų overhead'ą. Su 8 gijų gauname $\sim 6.85\times$ pagreitį ($E \approx 0.86$), o tai yra puikus rezultatas. Tačiau 16 gijų spartinimas siekia $\sim 8.8\times$ ($E \approx 0.55$), o 32 gijų – vos $\sim 8.7\times$. Kritimas po 8–16 gijų rodo, kad papildomos gijos nebeatneša proporcingo pagreičio dėl mažėjančio darbo vienam thread'ui, kitaip sakant nes kiekvienai gijai tenka vis mažiau darbo, o sinchronizacijai reikalingas laikas tampa jau reikšmingas, todėl bendras pagreitis nebeauga proporcingai.



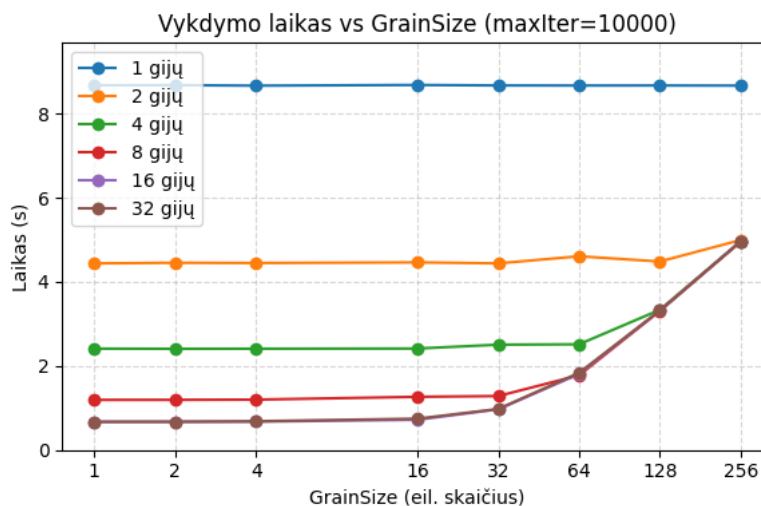
Efektyvumo $E(p)=S(p)/p$ grafikas parodo, kokią dalį idealaus pagreičio mes pasiekiamo. Iki 8 gijų efektyvumas yra aukštas (~ 85 – 95%), t. y. kiekviena gija gauna pakankamai darbo, kad sinchronizacijos laikas beveik nepastebimas.. Su 16 gijų E krenta iki $\sim 55\%$, o 32 gijų – iki $\sim 27\%$. Tai signalizuoja, jog didėjant gijų skaičiui darbo ir sinchronizacijos santykis smarkiai blogėja: gijos pradeda konkuruoti dėl atminties pralaidumo, o blokų skaičius kiekvienai gijai sumažėja, todėl overhead'as santykinai auga.

Atliekant antrą tyrimą, kur buvo fiksuotas vienos užduoties dydis eilučių atžvilgiu ($\text{grainSize} = 32$), rezultatai buvo tokie:

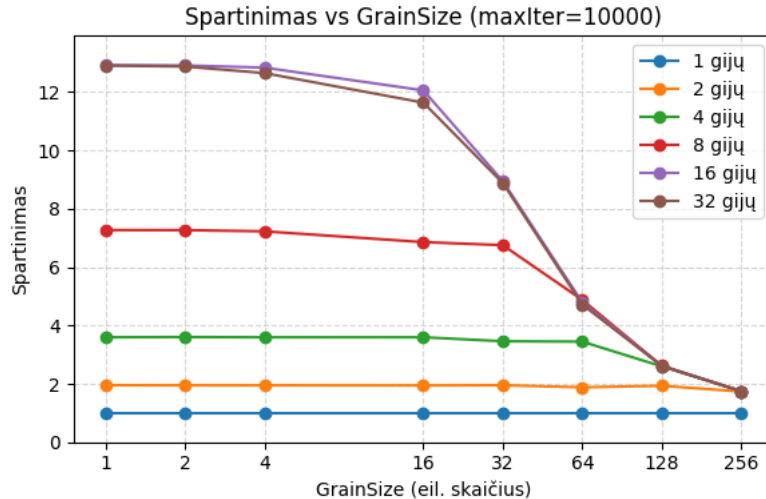


Šiame grafike rodomas spartinimo $S(p)$ santykis su gijų skaičiumi p kelioms problemos dydžio reikšmėms ($\text{maxIter} = 1000, 2000, 5000, 10000, 50000, 100000$). Visos kreivės smarkiai kyla iki 8 gijų ($S \approx 3.5\text{--}3.6\times$ su 4 gijom, $S \approx 6.8\text{--}6.9\times$ su 8), toliau pasiekia aukščiausią tašką $\sim 9\times\text{--}9.3\times$ su 16 gijų ir šiek tiek sumažėja su 32. Skirtingas maxIter ilgainiui duoda vos juntamą efektą: didesnių iteracijų atveju $S(16)$ truputį aukštesnis ($\sim 9.25\times$ vs $\sim 8.80\times$), nes daugiau skaičiavimo darbo geriau paslepia sinchronizacijos laika. Vis dėlto nė viena kreivė ne priartėja idealiai $S(p)=p$ linijai nes pasiektos branduolių ribos ir padidėjęs tarpusavio derinimo bei atminties dalijimosi laikas.

Atliekant trečią tyrimą, kur buvo fiksuotas iteracijų skaičius ($\text{maxIter} = 10000$), rezultatai buvo tokie:



Grafikas parodo, kiek laiko trunka viso paveikslų sugeneravimas (sekundėmis) keičiant darbo blokų aukštį ($\text{grainSize} = 1, 2, 4, 16, 32, 64, 128, 256$), tuomet kai naudojame 1, 2, 4, 8, 16 arba 32 gijas. Mažiausių blokų atveju ($\text{grainSize} 1\text{--}4$) laikas yra minimalus: $8.66\text{--}8.68$ s su 1 gija, $1.19\text{--}1.20$ s su 8 gijom ir $0.67\text{--}0.68$ s su 16 gijų. Tačiau didėjant grainSize virš 16, laikas greitai didėja, pavyzdžiui su 64 eilutėmis vienam blokui laikas padidėja iki 1.8 s (8 gijos) ar 4.6 s (2 gijos), o su 256 ~ 4.96 s su 2 gijomis. Tai reiškia, kad per stambius blokus darbų paskirstymas tampa labai netolygus, kai kurios gijos baigia anksčiau ir laukia, kol kitos baigs savo darbą, o sinchronizacijos kvietimų skaičius sumažėja be darbo balanso naudos.



Šiame grafike matome $S(p)=T(1)/T(p)$ kreives dar kartą visiems grainSize reikšmėms, kai gijų skaičius yra nuo 1 iki 32. Smulkus grainSize (1–4) duoda aukščiausią spartinimą – su 16 gijų net $\sim 12.9\times$; tai reiškia, kad darbo blokų dydis yra pakankamai mažas, kad gijos visada gautų naujų užduočių, o synchronizacijos kvietimų skaičius nėra pernelyg didelis. Tačiau esant $\text{grainSize} \geq 16$ –32 spartinimas ėmė smuktelėti (su 16 gijų S nuo $\sim 12.9\times$ iki $\sim 12.1\times$ ir toliau žemyn), o su $\text{grainSize} 128$ –256 S kritęs iki ~ 2 – $1.7\times$. Tokiu būdu matome, jog pernelyg stambūs blokai smarkiai prastina balansavimą ir išnaudoja gijų resursus neefektyviai.

Išvados

Atliekant eksperimentus su skirtingu gijų skaičiumi, matėme, kad Mandelbrot fraktalo generavimas labai gerai plečiasi iki maždaug fizinių branduolių skaičiaus ribos, tai yra 8–16 gijų. Iki 8 gijų spartinimas buvo $S(8)\approx 6.8\times$, o su 16 gijų pasiekėme maždaug $9\times$ pagreitį. Virš šios ribos, t. y. naudojant 32 gijas, spartinimas nebepagerėjo – jį riboja synchronizacijos ir atminties konkurencijos kaštai. Be to, didinant iteracijų ribą (maxIter) nuo 1 000 iki 100 000 pastebėta, kad didesnis darbo kiekis šiek tiek pagerina spartinimą aukštesnėse gijų, nes daugiau skaičiavimo užduočių geriau paslepia derinimosi laika.

Analizė su grainSize reikšmėmis nuo 1 iki 256 parodė, jog optimaliausias darbo blokų dydis yra smulkus – 1–4 eilučių vienam uždaviniui. Šiame intervale pasiekti geriausi laiko ir spartinimo rezultatai: minimalūs vykdymo laikai bei aukščiausias $S(p)$ ($\sim 12.9\times$ su 16 gijų). Padidinus grainSize virš 16, laikas pamažu augo, o spartinimas reikšmingai mažėjo, nes stambūs blokai prastino apkrovos balansavimą ir gijos ilgai laukdavo vienos kitų. Atsižvelgiant į tai, galima naudoti 8–16 gijų, $\text{maxIter} \geq 10\,000$ ir $\text{grainSize} \approx 1$ –4, kad būtų pasiektas optimalus rezultatas.

